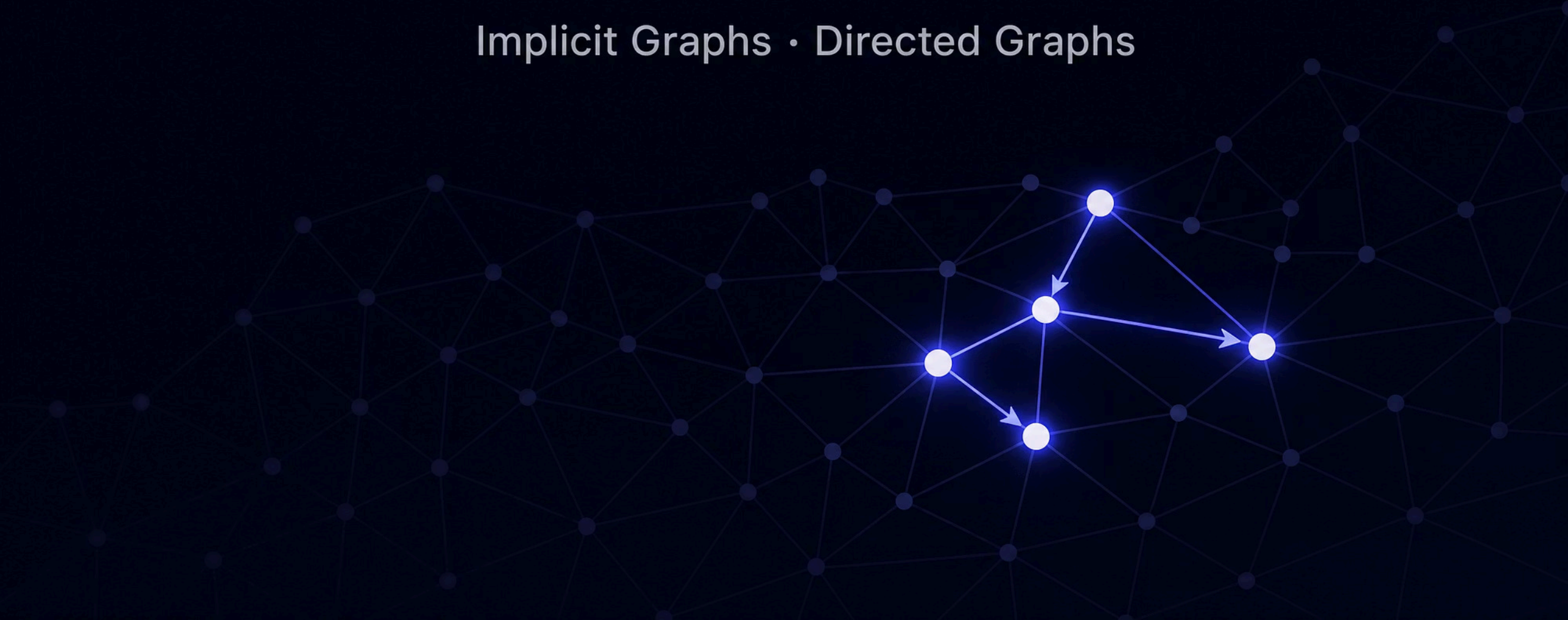


Graph Patterns — Advanced Module 1

Implicit Graphs · Directed Graphs



Topics Covered

1

Implicit Graphs

2

Directed Graphs



Part 1: Implicit Graphs — Topic Classification

 Category:	Graph Algorithms / Problem Solving Patterns
 Difficulty Level:	Advanced
 Prerequisites:	Graph Basics, BFS, DFS, Hash Maps, Recursion
 Industry Relevance:	Critical — Powers AI state-space search, puzzle solvers, compiler optimisation, network simulation, game engines
 Recommended Path:	Graph Basics → BFS → DFS → Matrix as Graph → Implicit Graphs → Dijkstra → A*

What Is an Implicit Graph?

An implicit graph is a graph $G = (V, E)$ where vertices and edges are not stored explicitly in memory but are instead generated on demand from a compact problem representation. The graph may be infinite or exponentially large, making explicit construction impossible or impractical.

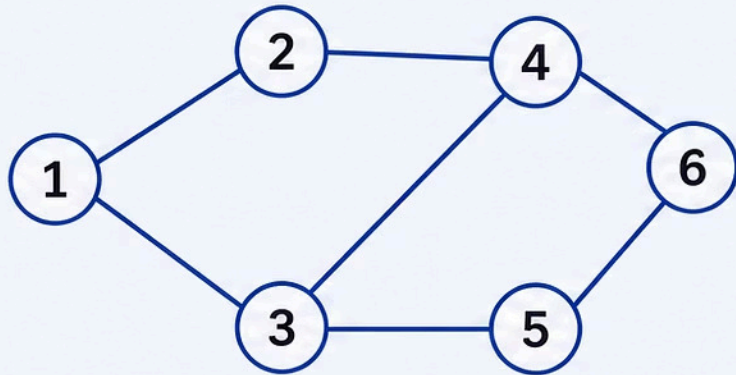
Formally defined by: an initial state (the source vertex), a neighbor-generation function $\text{neighbors}(\text{state}) \rightarrow \text{list of } (\text{next_state}, \text{cost})$, and a goal test $\text{is_goal}(\text{state}) \rightarrow \text{bool}$.



The Core Insight

Explicit Graph vs. Implicit Graph

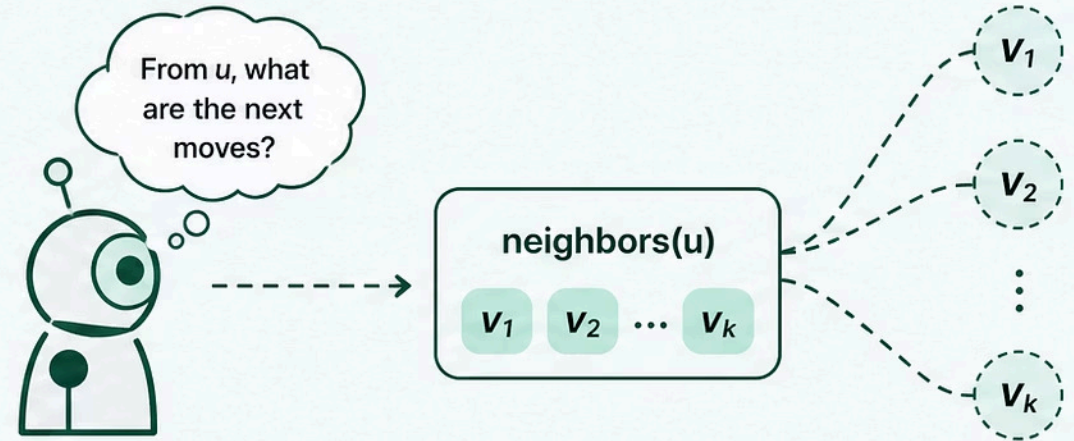
Explicit: Pre-built, Labelled Map



adj

1: [2, 3]
2: [1, 4]
3: [1, 4, 5]
4: [2, 3, 6]
5: [3, 6]
6: [4, 5]

Implicit: Generate Next Step on the Fly



- 1 **Explicit:** Store $\text{adj}[u] = [v_1, v_2]$ — Build before searching — Memory: $O(V + E)$
- 2 **Implicit:** Define $\text{neighbors}(u)$ — Generate v_1, v_2 on the fly — Memory: $O(\text{current frontier})$
- 3 When $|V| = 10^{20}$ (8-puzzle permutations, chess positions): Explicit: impossible. Implicit: tractable with BFS/DFS.
- 4 Every BFS or DFS on an implicit graph visits only the reachable portion — potentially a tiny fraction of the conceptual full graph.

Why Does It Exist? — The Explosion Problem

For many real problems, explicitly enumerating all vertices is infeasible:

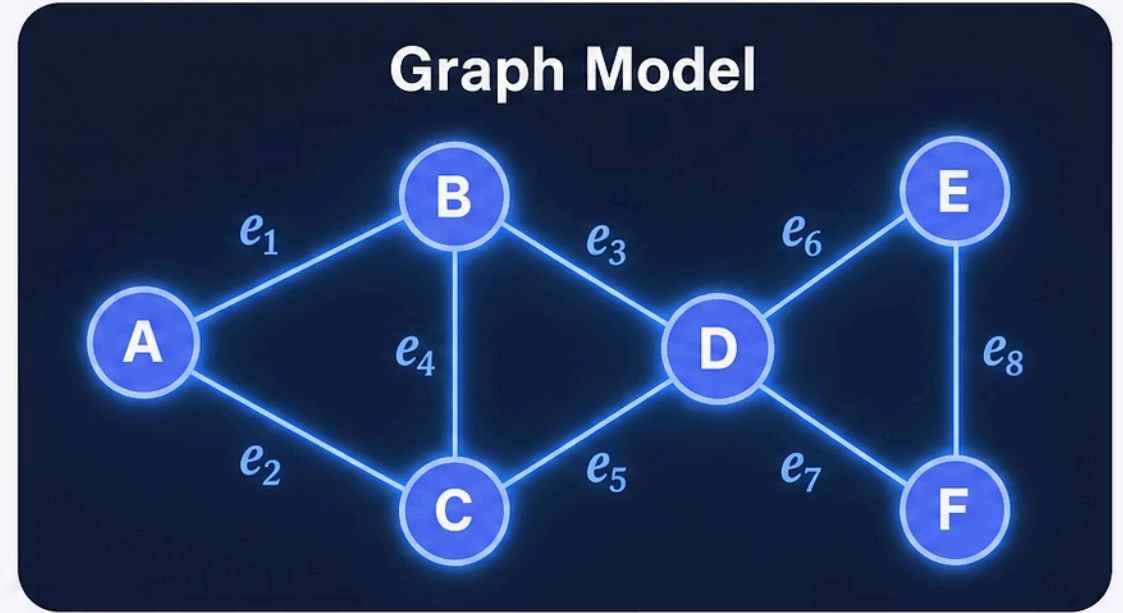
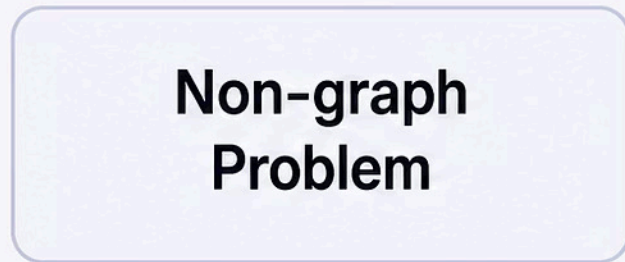
- 8-puzzle (3×3): $9!/2 = 181,440$ — Borderline
- 15-puzzle (4×4): $16!/2 \approx 10^{13}$ — ✗ No
- Chess positions: $\approx 10^{43}$ — ✗ No
- Rubik's Cube: $\approx 4.3 \times 10^{19}$ — ✗ No
- Protein folding states: Astronomical — ✗ No

Implicit graphs allow us to search these spaces without materialising them.



The Transformation Pattern

The key engineering skill: transform a non-graph problem into a graph problem by defining states and transitions.



1

Word ladder: Vertices = words; Edges = single-char differences; BFS

2

Minimum coin changes: Vertices = amounts 0..target; Edges = subtract each coin; BFS

3

Shortest path in maze: Vertices = (row, col) pairs; Edges = valid moves; BFS

4

Collect all keys: Vertices = (pos, keys_held) bitmask; Edges = moves that pick up keys; BFS

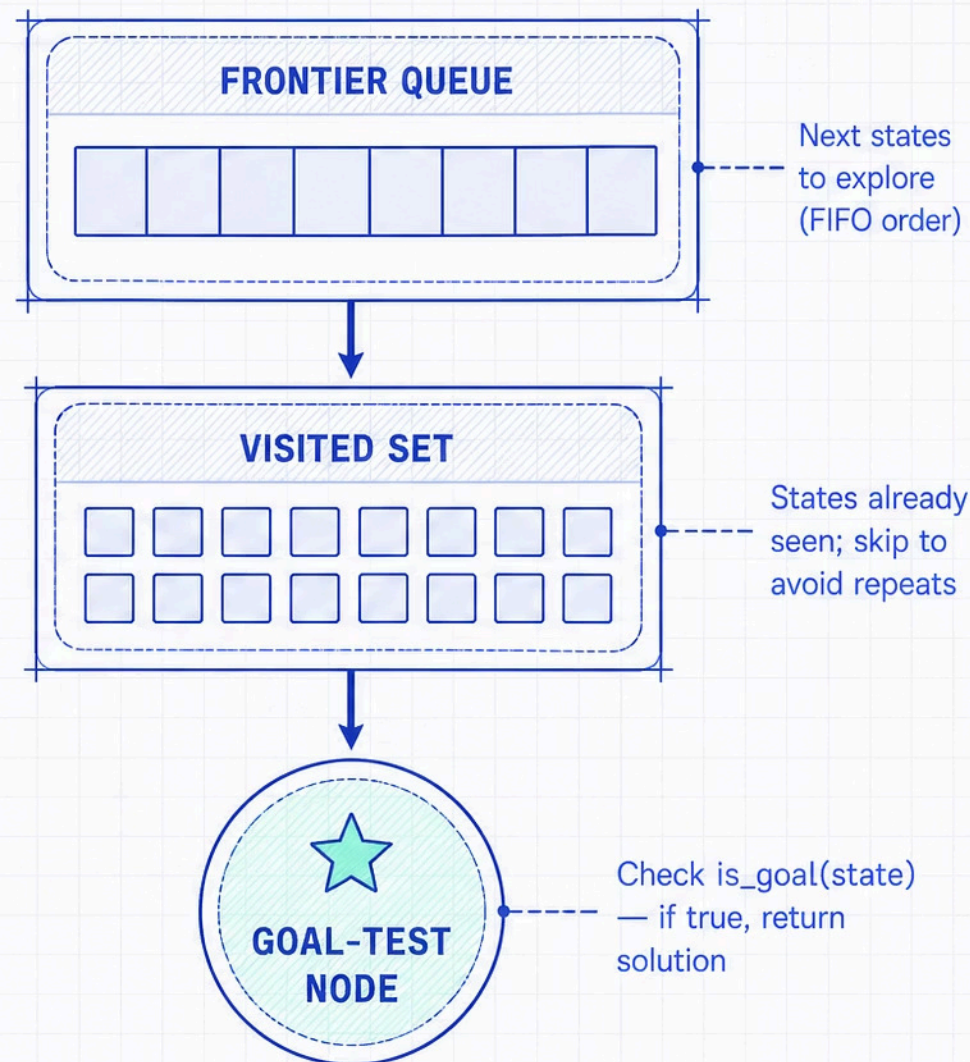
The Implicit Graph Template

State Design — The Critical Skill

What constitutes a "state" must capture everything needed to determine future moves. Including too little → incorrect. Including too much → too many states → TLE.



1. **WRONG** state (keys problem): (row, col) — doesn't encode which keys are held
2. **CORRECT** state: (row, col, frozenset(keys_held)) — full context; correct answer possible
3. **OPTIMISED** state: (row, col, bitmask_of_keys) — $O(1)$ operations; hashable
4. The **ONLY** things that change between problems: state representation, get_neighbors(), is_goal()

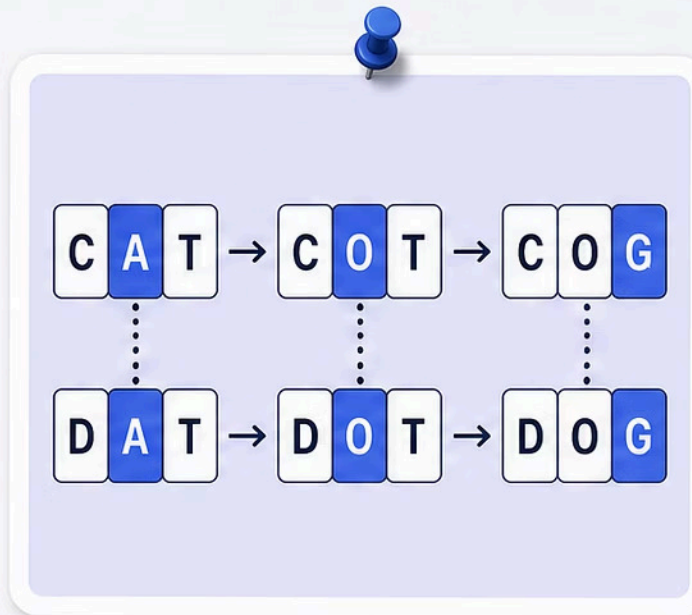


Patterns 1–3: BFS Variants



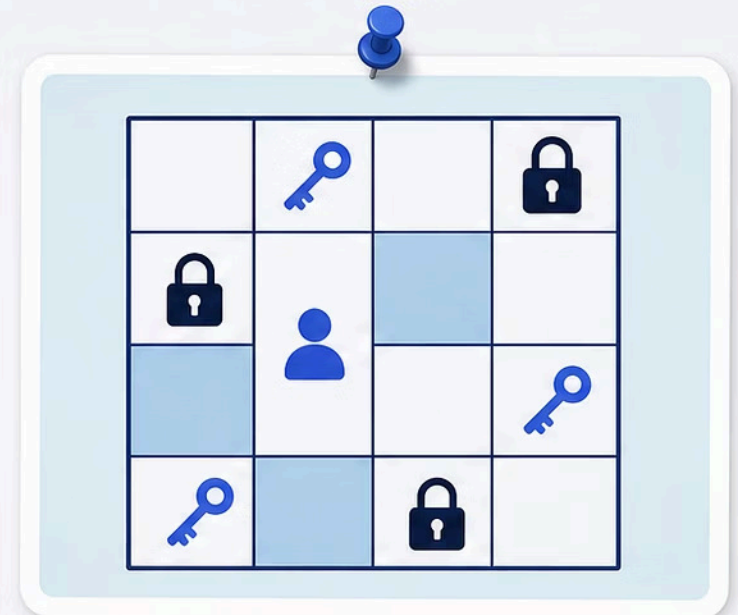
1 Pattern 1 — BFS on Number/Amount States: Water Jug Problem.

- State = (amount_in_x, amount_in_y). Neighbors = fill/empty/pour transitions.



2 Pattern 2 — BFS on String/Configuration States: Word Ladder.

- State = string. Neighbors = all single-character substitutions in word_set.
- Time: $O(N \times L \times 26)$.



3 Pattern 3 — BFS on Compound States (Position + Inventory): Shortest path collecting all keys.

- State = (row, col, keys_bitmask).
- $V = \text{ROWS} \times \text{COLS} \times 2^{\text{num_keys}}$.

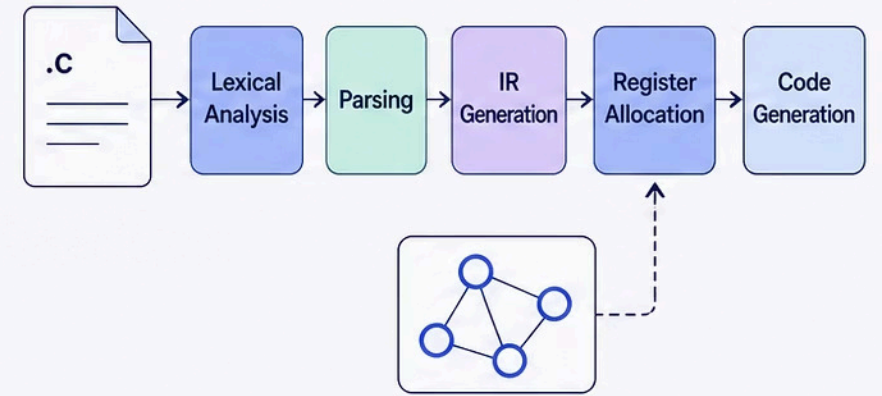
Patterns 4–5:

Weighted & Bidirectional

- 1 Pattern 4 — Dijkstra on Implicit Weighted Graph:** When transition costs vary, replace BFS with Dijkstra's priority queue. Example: minimum cost to mutate a protein sequence.
- 2 Pattern 5 — Bidirectional BFS:** When both source and target are known, bidirectional BFS dramatically reduces the search space. Reduces search from $O(b^d)$ to $O(b^{(d/2)})$. Classic use: open lock problem, word ladder with known target.



Real-World Use Cases



- 1 **AI Game Playing — Puzzle Solvers:** IDA*, A*, 8-puzzle/15-puzzle. A* explores ~1000 nodes to solve any 15-puzzle vs BFS's 10^{13} states.
- 2 **Network Protocol Simulation:** AWS uses TLA+ to model-check S3, DynamoDB — BFS on implicit state graphs of distributed protocol configurations.
- 3 **Compiler Register Allocation:** LLVM, GCC backend. DFS with backtracking on implicit graph. Full assignment space is exponential; only valid partial assignments are explored.

Implementation:

Open Lock (LeetCode #752)

4-digit lock, each digit 0–9. Start: "0000". Target: given string. Dead-ends: set of forbidden combinations. Minimum turns to reach target.

- 1 State: 4-digit string ("0000" to "9999") — $10^4 = 10,000$ states
- 2 Neighbors: 8 per state (each of 4 digits $\pm 1 \bmod 10$)
- 3 BFS guarantees minimum turns



Hands-On Labs

Lab 1: Minimum Steps to Target Sum

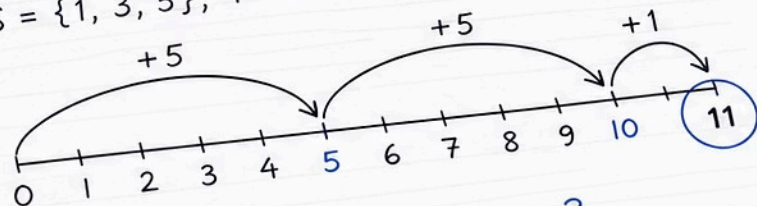
Start at 0; add any number from set S ; find minimum steps to reach target T . Test: $S=\{1,3,5\}$, $T=11 \rightarrow$ expected: 3 steps ($5+5+1$ or $5+3+3$). Compare with greedy — $S=\{1,3,4\}$, $T=6$: greedy gives $4+1+1=3$ but BFS finds $3+3=2$.

Lab 2: Word Ladder II — All Shortest Paths

BFS layer by layer; record all parents; reconstruct all paths via DFS on parent map. Test: $\text{beginWord}=\text{"hit"}$, $\text{endWord}=\text{"cog"}$. Expected: $[\text{"hit"}, \text{"hot"}, \text{"dot"}, \text{"dog"}, \text{"cog"}]$ and $[\text{"hit"}, \text{"hot"}, \text{"lot"}, \text{"log"}, \text{"cog"}]$.

Minimum Steps to Target Sum

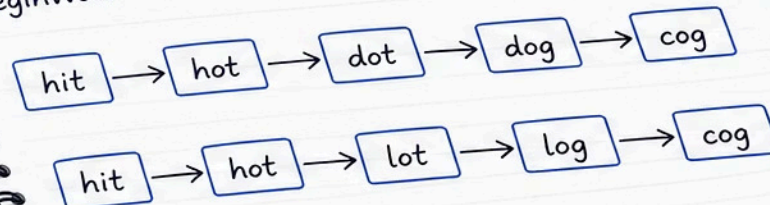
$$S = \{1, 3, 5\}, T = 11$$



$$\text{Steps: } 5 + 5 + 1 = 3$$

Word Ladder II — All Shortest Paths

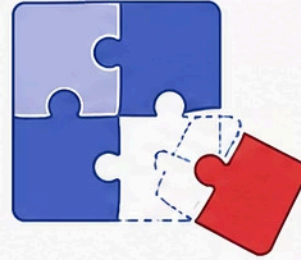
$\text{beginWord} = \text{"hit"}, \text{endWord} = \text{"cog"}$



Common Mistakes & Pitfalls — Implicit Graphs

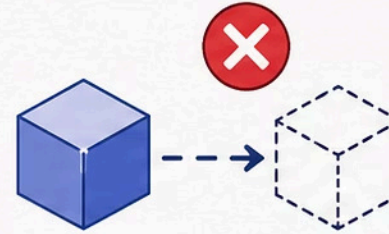
- 1 State doesn't capture full context → wrong answers. Fix: include all context in state.
- 2 Using mutable objects as state → unhashable. Fix: use tuples, frozensets, integers (bitmasks).
- 3 Not pruning visited states → exponential revisiting. Fix: always check and add to visited BEFORE enqueueing.
- 4 Choosing BFS when weights vary → wrong shortest path. Fix: use Dijkstra when edge costs differ.
- 5 State space too large without heuristic → BFS times out. Fix: use A* with admissible heuristic; bidirectional BFS; IDA*.
- 6 Forgetting goal is also a state → miss the answer. Fix: check goal at enqueue and dequeue.

1



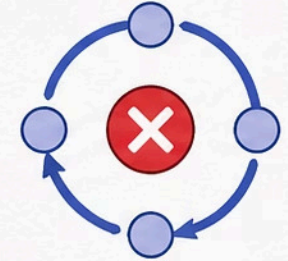
State doesn't capture full context → wrong answers.
Fix: include all context in state.

2



Using mutable objects as state → unhashable.
Fix: use tuples, frozensets, integers (bitmasks).

3



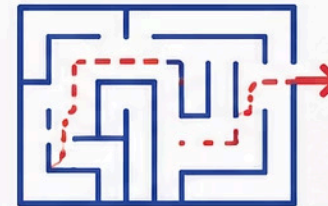
Not pruning visited states → exponential revisiting.
Fix: always check and add to visited BEFORE enqueueing.

4



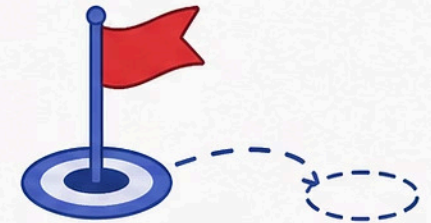
Choosing BFS when weights vary → wrong shortest path.
Fix: use Dijkstra when edge costs differ.

5



State space too large without heuristic → BFS times out.
Fix: use A* with admissible heuristic; bidirectional BFS; IDA*.

6



Forgetting goal is also a state → miss the answer.
Fix: check goal at enqueue and dequeue.

Learning Summary — Implicit Graphs

1

An implicit graph defines vertices/edges via a `neighbors(state)` function — never built explicitly

2

State design is the hardest part: must capture all decision-relevant context without being unnecessarily large

3

Use bitmasks for inventory/flag states — compact, hashable, $O(1)$ operations

4

BFS → uniform cost (min steps); Dijkstra → variable cost; A* → large space with heuristic

5

Bidirectional BFS reduces search from $O(b^d)$ to $O(b^{(d/2)})$ when target is known

6

The neighbor function IS the graph — changing it changes the entire problem structure



Part 2: Directed Graphs

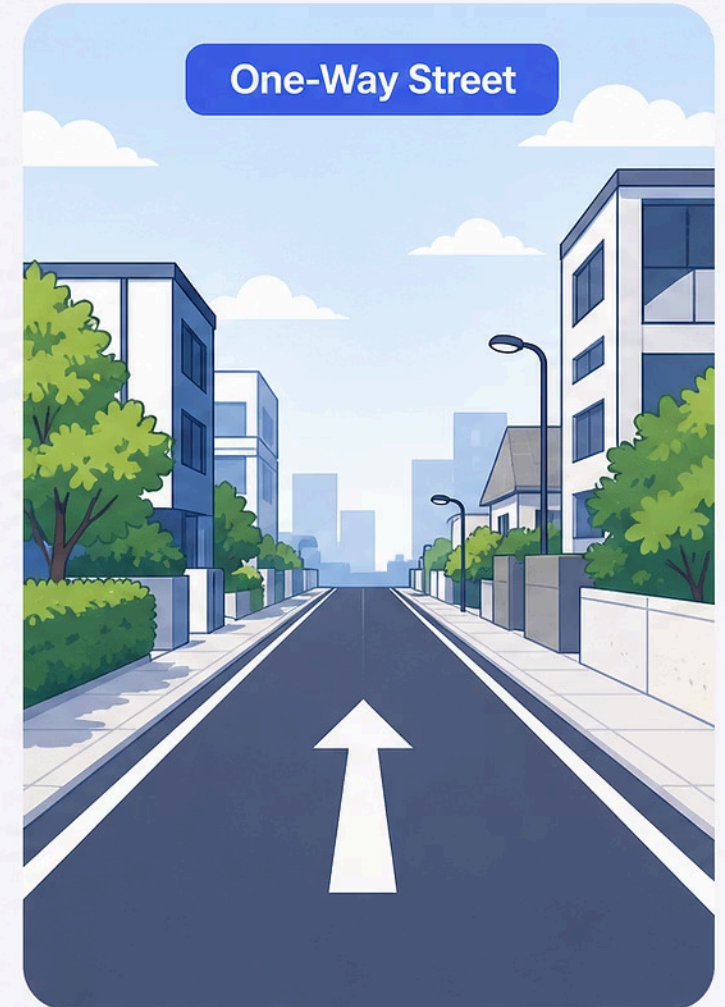
Digraphs — Dependency systems, compilers, web graphs, distributed systems, scheduling, social networks

Recommended Path: Graph Basics → BFS → DFS → Directed Graphs → Topological Sort → SCC

What Is a Directed Graph?

A directed graph (digraph) $G = (V, E)$ is a graph where every edge has a direction: an edge (u, v) means "from u to v " and does NOT imply (v, u) exists. Directed edges are called arcs.

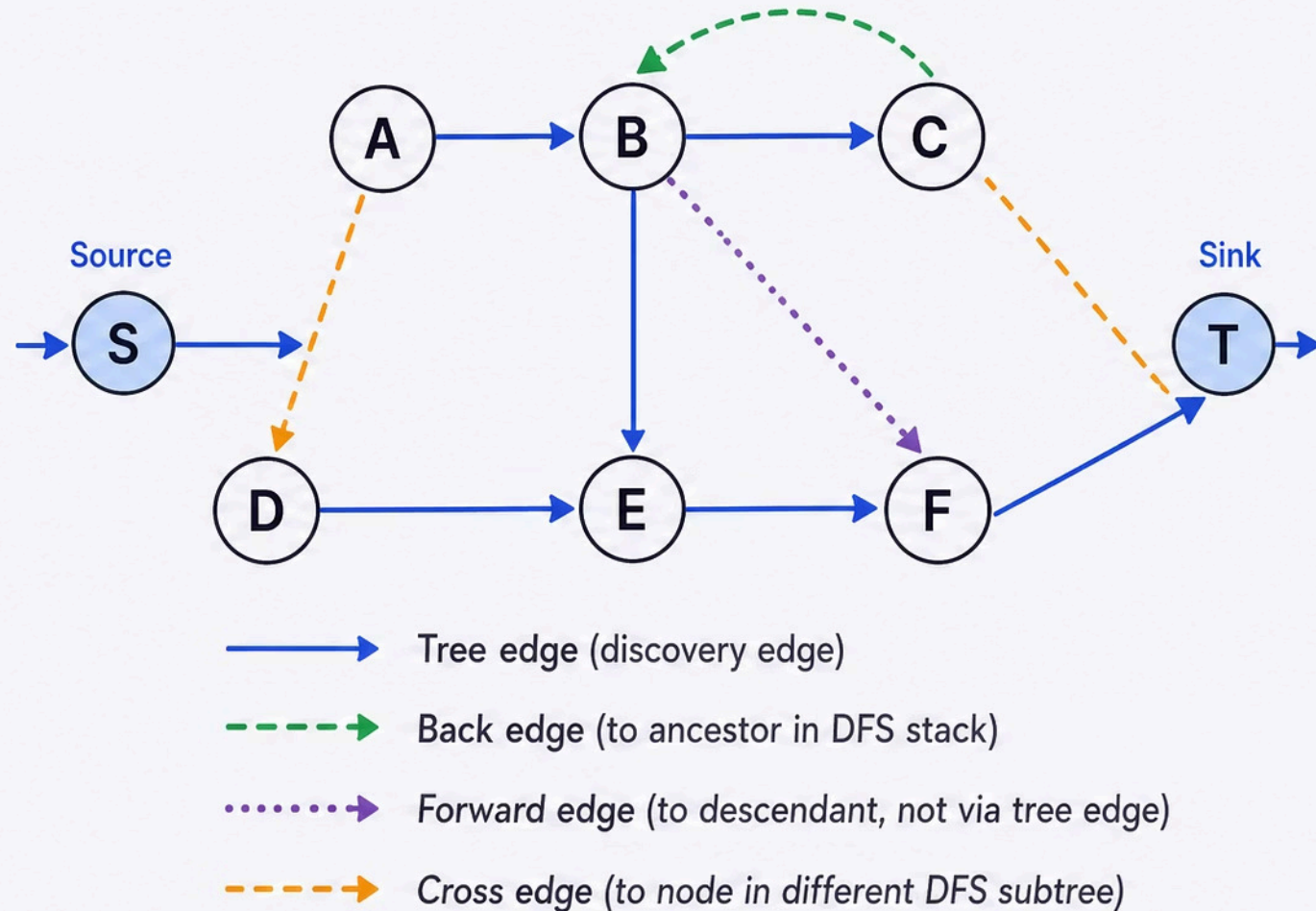
- Undirected: Facebook friends, two-way road, mutual dependency
- Directed: Twitter follows, one-way street, A depends on B (not vice versa)
- $\text{adj}[u]$ contains v AND $\text{adj}[v]$ contains u (undirected) vs. $\text{adj}[v]$ does NOT contain u unless explicitly added (directed)
- Reachability is asymmetric: if u can reach v , it does NOT mean v can reach u



Key Structural Concepts

In-degree, Out-degree, DFS Edge Types

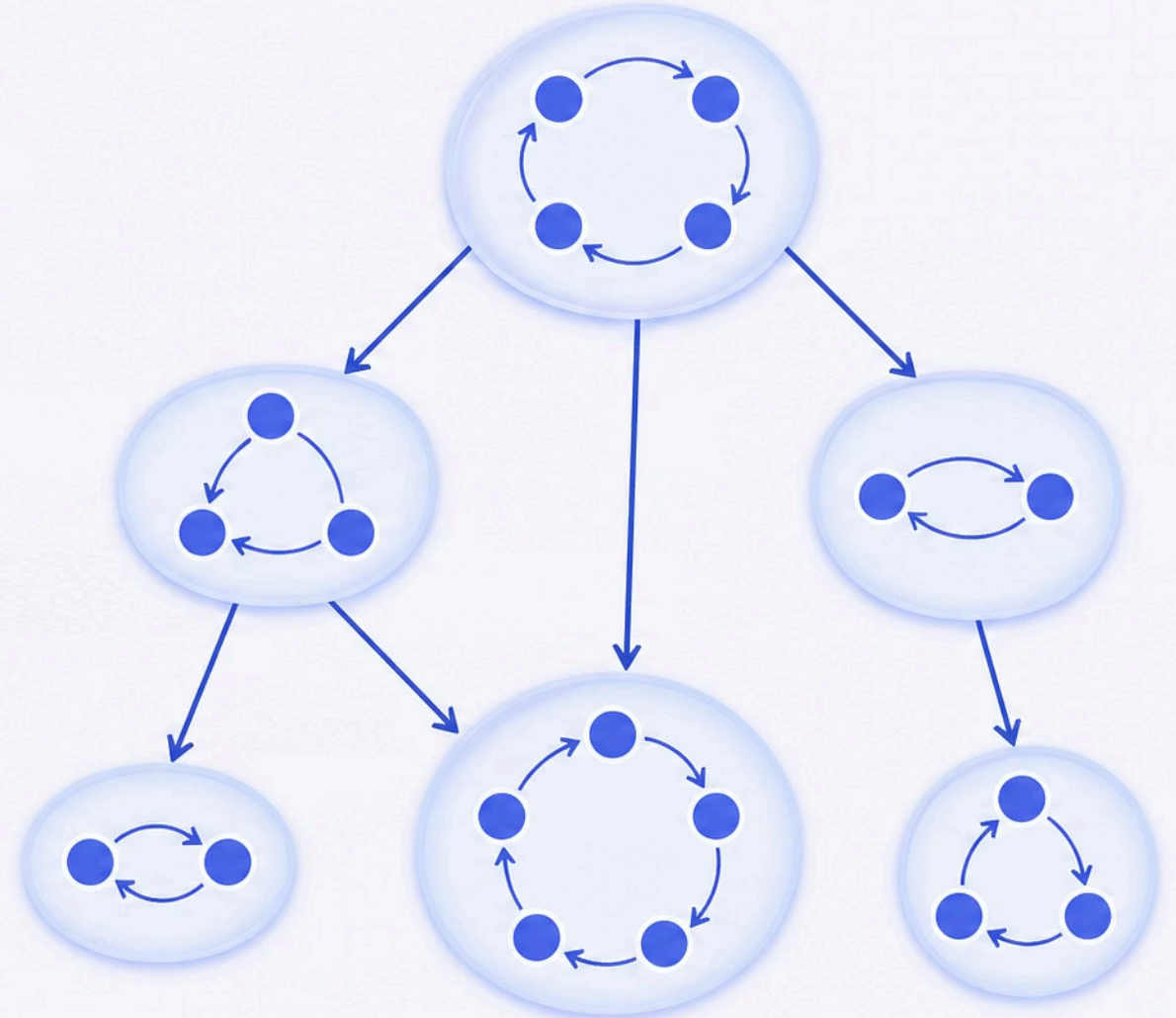
- 1 $\text{in_degree}(v)$ = edges pointing INTO v ;
 $\text{out_degree}(v)$ = edges pointing OUT OF v
- 2 Identity: $\sum \text{in_degree}(v) = \sum \text{out_degree}(v) = |E|$.
Source: $\text{in_degree}=0$. Sink: $\text{out_degree}=0$.
- 3 Tree edge: v was unvisited. Back edge: v in current DFS stack $\rightarrow$ CYCLE. Forward edge: v is descendant not via direct tree edge. Cross edge: v in different DFS subtree.
- 4 Key: Only back edges indicate cycles in directed graphs. Forward and cross edges do NOT create cycles.



Strongly Connected Components (SCCs)

A Strongly Connected Component is a maximal set of vertices S such that for every pair (u, v) in S , there exists a path $u \rightarrow \dots \rightarrow v$ AND $v \rightarrow \dots \rightarrow u$.

- 1 Condensation DAG: SCC graph is always a DAG — guaranteed no cycles between SCCs
- 2 Tarjan's SCC: single DFS pass; $\text{low}[u] == \text{disc}[u] \rightarrow$ SCC root. Time: $O(V+E)$. Industry-standard.
- 3 Kosaraju's: two DFS passes; second on transpose graph. Time: $O(V+E)$. Easier to implement.
- 4 Functional graphs (out_degree=1 always): exactly one cycle per component — use Floyd's Tortoise and Hare. $O(V)$ time.



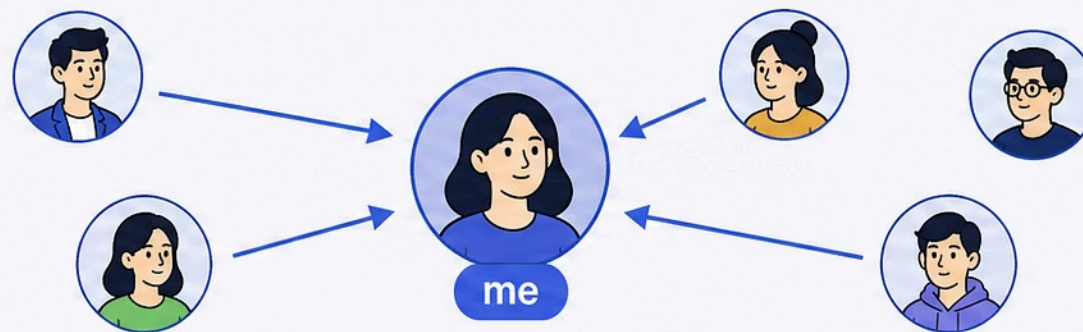
Real-World Use Cases — Directed Graphs

- 1. Google PageRank:** web pages = vertices;
hyperlinks = directed edges.
$$PR(v) = (1-d)/N + d \times \sum (PR(u) / \text{out_degree}(u))$$

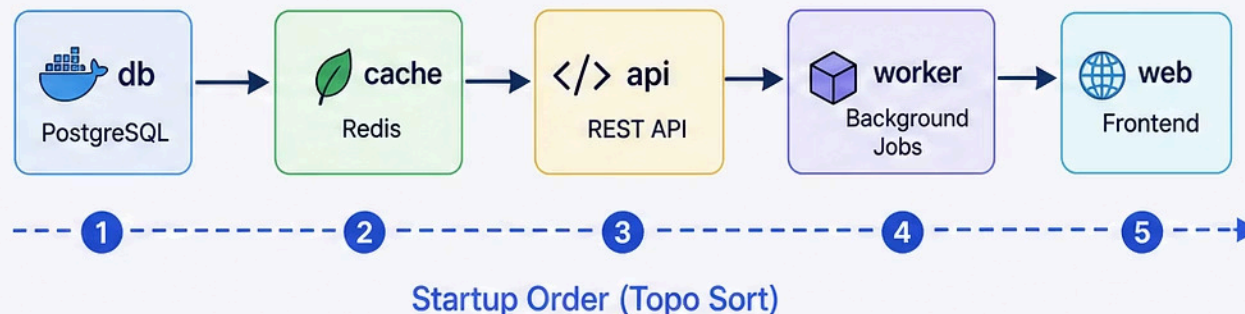
for all $u \rightarrow v$ edges. In-links boost rank;
out-links dilute.



- 2. Twitter/Instagram Follow Graph:**
"Who follows me?" = $\text{in_neighbors}(\text{me})$.
GraphJet processes directed follow-graph queries in real time for feed generation and recommendation.



- 3. Kubernetes/Docker Dependency Resolution:**
services = vertices; depends_on = directed edges.
Topo sort gives startup order; cycle detection identifies invalid configurations.



Learning Summary — Directed Graphs

KEY TAKEAWAYS

- 1 Directed graphs model asymmetric relationships; edge $(u \rightarrow v)$ does not imply $(v \rightarrow u)$
- 2 In-degree and out-degree are separate; $\sum \text{in_deg} = \sum \text{out_deg} = |E|$
- 3 DFS produces 4 edge types in digraphs: tree, back (cycles!), forward, cross
- 4 SCCs: maximal sets of mutually reachable vertices; found in $O(V+E)$ via Tarjan's or Kosaraju's
- 5 Condensation of any directed graph is always a DAG — this is the key to simplification
- 6 Functional graphs (out_degree=1 always) have exactly one cycle per component — use Floyd's
- 7 DAGs are the most tractable digraph class: topo sort + DP work cleanly on them

CHEAT SHEET

- **Digraph:** Vertices V , directed edges $E \subseteq V \times V$
- **Edge $(u \rightarrow v)$:** Direction matters; not symmetric
- **Degrees:** $\text{in_deg}(v)$, $\text{out_deg}(v)$; $\sum \text{in_deg} = \sum \text{out_deg} = |E|$
- **DFS Edge Types:** tree, back (cycles!), forward, cross
- **SCCs:** Maximal mutually reachable sets; found in $O(V+E)$ via Tarjan's or Kosaraju's
- **Condensation Graph:** SCCs contracted $\rightarrow$ always a DAG
- **Functional Graphs:** out_degree = 1 always; exactly one cycle per component; use Floyd's
- **DAG Strength:** Topological order exists; topo sort + DP work cleanly